📄 Week 12 – DevSecOps Hands-on: Security Automation

---

1. Introduction

Modern DevOps practices require not only automation but also strong security integration. DevSecOps ensures that security is embedded into the development lifecycle, enabling teams to identify and address vulnerabilities early.

This session focused on applying DevSecOps principles in practice by demonstrating how vulnerabilities can be detected, resolved, and prevented using automated pipelines.

---

2. Objective of the Hands-on

The objective of this session was to provide practical exposure to:

- Identifying vulnerabilities in application dependencies
- Understanding severity and impact of security issues
- Applying remediation techniques
- Integrating security checks into CI/CD pipelines
- Preventing insecure code from progressing through the pipeline

---

3. Vulnerability Scanning

Vulnerability scanning is the process of analyzing application dependencies to detect known security issues.

In the demonstration, a dependency with known vulnerabilities was introduced into the application. This allowed learners to observe how security tools can detect risks even when the application code itself appears correct.

Key Observations

- Vulnerabilities can exist in third-party libraries

- Security risks are categorized by severity levels such as low, moderate, high, and critical
- Even simple applications may contain serious vulnerabilities

---

4. Vulnerability Remediation

After identifying vulnerabilities, the next step is remediation.

During the session, vulnerabilities were resolved by updating or replacing affected dependencies. This demonstrated how security issues can be addressed without modifying core application logic.

Key Observations

- Fixing vulnerabilities is essential before deployment
- Automated tools can assist in applying safe updates
- Security is a continuous process, not a one-time activity

---

5. Security Integration in CI/CD Pipeline

To prevent vulnerabilities from being overlooked, security checks must be integrated into the CI/CD pipeline.

In the demonstration, a security scanning step was added to the pipeline. This ensured that every code change triggered an automated vulnerability check.

Key Observations

- CI pipelines can enforce security standards
- Security checks can automatically validate dependencies
- Builds can be configured to fail if vulnerabilities exceed a defined severity level

---

6. Preventing Security Regressions

A critical aspect of DevSecOps is ensuring that previously resolved vulnerabilities are not reintroduced.

This was demonstrated by intentionally adding a vulnerable dependency after implementing pipeline security checks.

Outcome

- The pipeline failed when insecure code was introduced
- The system prevented progression of vulnerable code
- Once the issue was resolved, the pipeline executed successfully

Key Observations

- Security must be enforced continuously
- Automated checks prevent human error
- CI/CD pipelines act as a security gate

---

7. DevSecOps Workflow

The complete workflow demonstrated during the session can be summarized as:

Code → Vulnerability Detection → Remediation → CI Security Check → Secure Build → Deployment

This workflow ensures that only secure and validated code is allowed to move forward.

---

8. Key Concepts Covered

- Dependency-based vulnerability detection
- Severity-based risk classification
- Remediation of security issues
- CI/CD-based security enforcement
- Prevention of insecure deployments

---

9. Summary

This hands-on session demonstrated how DevSecOps practices enhance application security by integrating automated checks into the development workflow.

Key takeaways include:

- Security vulnerabilities can originate from dependencies
- Automated tools help identify and resolve issues efficiently
- CI/CD pipelines can enforce security policies
- Secure development practices reduce risks in production environments

---

Assignment – DevSecOps Implementation

Objective

To apply DevSecOps principles by identifying, resolving, and preventing vulnerabilities within an application using CI/CD automation.

Tasks

Part 1: Vulnerability Identification

- Analyze the application dependencies
- Identify existing vulnerabilities and their severity levels

---

Part 2: Vulnerability Remediation

- Resolve identified vulnerabilities
- Ensure that the application is free from critical issues

Part 3: CI/CD Security Integration

- Integrate vulnerability scanning into the CI/CD pipeline
- Ensure that security checks are automatically executed

Part 4: Security Enforcement Validation

- Introduce a vulnerable dependency intentionally
- Observe and verify pipeline failure
- Restore system by resolving the vulnerability

Deliverables

- GitHub repository link
- Updated CI/CD pipeline configuration
- Evidence of:
    - Vulnerability detection
    - Successful remediation
    - Pipeline failure due to security issue
    - Pipeline success after resolution

Evaluation Criteria

Criteria	Description
Vulnerability Awareness	Ability to identify security issues
Remediation	Proper resolution of vulnerabilities
CI/CD Integration	Security checks implemented
Pipeline Enforcement	Pipeline blocks insecure code
Recovery	System restored after fixing issues

Note

The assignment is designed to simulate real-world DevSecOps workflows where security is continuously enforced through automation. Learners are expected to demonstrate both understanding and practical implementation of secure development practices.